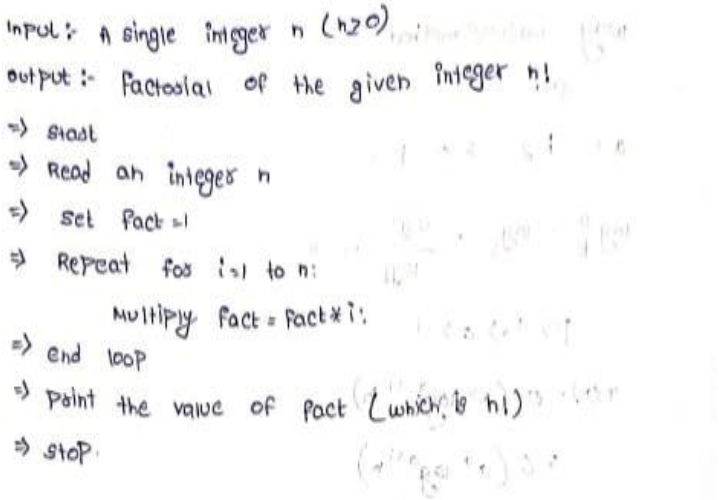
PRACTICAL-04

AIM: Implementation and Time analysis of factorial program using iterative and recursive method.

FACTORIAL BY ITERATIVE

ALGORITHM:



PROGRAM:

```
#include <iostream>
using namespace std;
int factorial_iterative(int n) {
    int fact = 1;
    for (int i = 1; i <= n; i++) {
        fact =fact*i;
    }
    return fact;
}

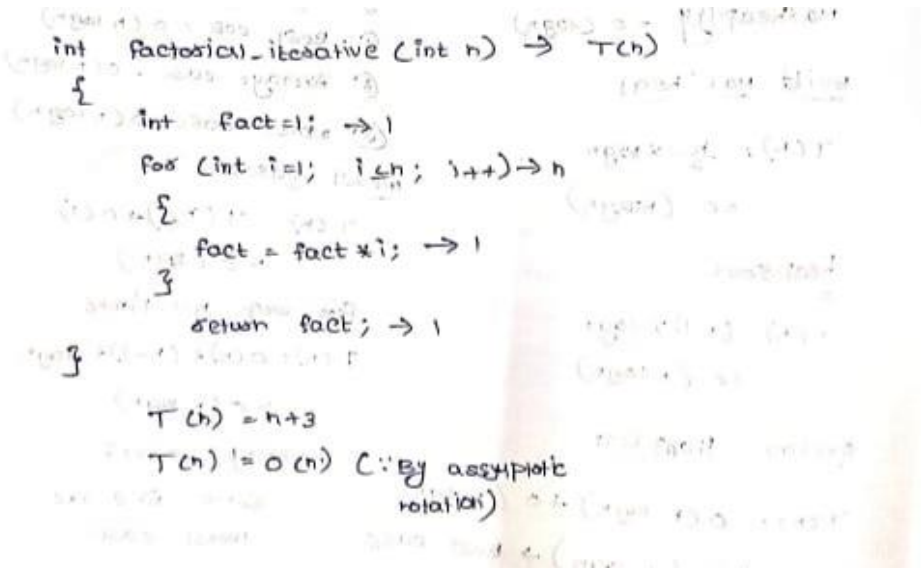
int main() {
```

```
    int n;
    cout << "Enter a number: ";
    cin >> n;
    cout << "Factorial (Iterative): " << factorial_iterative(n) << endl;
    return 0;
}
```

OUTPUT:

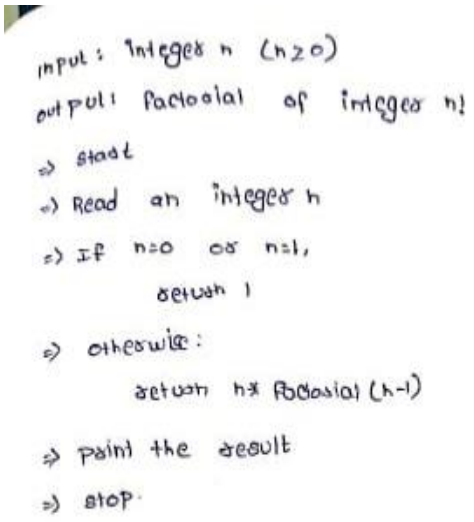
```
Enter a number: 5
Factorial (Iterative): 120
```

TIME COMPLEXITY:



FACTORIAL BY RECURSIVE

ALGORITHM:



PROGRAM:

```
#include <iostream>
using namespace std;
int factorial_recursive(int n) {
    if (n == 0 || n == 1)
        return 1;
    else
        return n * factorial_recursive(n - 1);
}
int main() {
    int n;
    cout << "Enter a number: ";
    cin >> n;
    cout << "Factorial (Recursive): " << factorial_recursive(n) << endl;
    return 0;
}
```

OUTPUT:

```
Enter a number: 5
Factorial (Recursive): 120
```

TIME COMPLEXITY:

```
int factorial_recursive (int n) -> T(n)
{
    if (n==0 || n==1) -> 1
        return 1;
    else:
        return n* factorial_recursive(n-1)
                  -> T(n-1)
```

$T(n) = T(n-1) + O(1)$

$= T(n-1) + 1$

$T(n) = T(n-1) + 1 \rightarrow (1)$

$n \rightarrow n-1$ put in (1)

$T(n-1) = T(n-1-1) + 1$

$= T(n-2) + 1 \rightarrow (2)$

Substitute (2) in (1)

$T(n) = T(n-2) + 1 + 1$

$T(n) = T(n-2) + 2 \rightarrow (3)$

$n \rightarrow n-2$ put in (3)

$T(n-2) = T(n-2-2) + 2$

$= T(n-4) + 2 \rightarrow (4)$

Sub (4) in (3)

$T(n) = T(n-4) + 2 + 2$

$T(n) = T(n-4) + 4 \rightarrow (5)$

$n \rightarrow n-4$ put in (5)

$T(n-4) = T(n-4-4) + 4$

$= T(n-8) + 4 \rightarrow (6)$

Substitute (6) in (5)

$T(n) = T(n-8) + 4 + 4$

$T(n) = T(n-8) + 8$

$\therefore T(n) = T(n-k) + k$

$= T(n-n) + n$

$= T(0) + n = 1 + n$

$[T(n) = 0, \; n-k = 0, \; n = k / k = n]$

$O(n) \rightarrow$ time complexity

CONCLUSION:

The factorial program demonstrates both iterative and recursive approaches. Iterative factorial runs in linear time $O(n)$with constant space, making it efficient. Recursive factorial also has $O(n)$time but requires $O(n)$stack space due to repeated function calls. Thus, iteration is more memory-efficient, while recursion is conceptually elegant.